The Math of Logic

Professor Logic's Workbook

One operator. Every formal system. No memorisation required.

James Alexander Pugmire · Propagation Logic Project · 2026

Illustrated by a chalkboard and a great deal of enthusiasm

BEFORE WE BEGIN

A Note from Professor Logic

PROFESSOR LOGIC

Hello. I am very glad you picked this up.

I want to tell you what this book is and what it is not. It is not a logic textbook in the traditional sense. Traditional logic textbooks hand you a list of rules, explain which symbols mean what, and ask you to apply the rules correctly. They treat logic as a given: here are the laws, here is how to use them, here is your grade.

This book does something different. It asks where the laws come from.

The answer turns out to be surprisingly simple. Logical laws are not chosen by committees of philosophers. They are forced out of arithmetic. You pick a collection of values, you pick three operations to work with them, and the laws that hold are the ones the arithmetic makes unavoidable. Change the values and different laws become unavoidable. The mechanism never changes.

That mechanism is what we are going to study. And we are going to study it the only way that actually works: by building things, testing them, and seeing what happens.

There will be a chalkboard. There will be exercises. There will be moments where I ask you to stop and think before reading on. Those moments are not optional extras. They are the actual lesson.

If you work through every exercise, you will finish this book able to look at any formal system in mathematics, logic, or computer science and immediately identify its moving parts. That skill will serve you for the rest of your mathematical life.

One more thing. I have an idea I want to hold onto for later: a "distinction operation" that might capture the cost of telling two values apart. We will come back to it after we have the foundations in place. For now, just tuck it in the back of your mind.

Right. Let us begin.

CHAPTER 1

What Can Something Be?

The question that starts all of mathematics.

PROFESSOR LOGIC

I want to start with a light switch. Not because light switches are particularly important in the grand scheme of things, but because a light switch is the simplest possible object that has what mathematicians call a *value*. And value is where everything begins.

A light switch can be on or it can be off. Those are its options. There is nothing else it can be. It cannot be "slightly on." It cannot be "on-ish." It is either one thing or the other.

Before you do any mathematics with any object, you need to know its options. What can it be? What is allowed? What is not?

The complete collection of things something is allowed to be has a name. I want to write it on the board.

CHALKBOARD

The carrier set *(also: value space)*

```
The complete collection of values an object is allowed to take.
Written using curly braces:   {   }   means "this is a collection".

Light switch:   { ON, OFF }   or equivalently   { 1, 0 }
Traffic light:   { RED, YELLOW, GREEN }   or   { 0, 0.5, 1 }
Coin:   { HEADS, TAILS }   or   { 0, 1 }
Your age:   { 0, 1, 2, 3, ... }   — infinitely many options
Temperature:   all real numbers   — the whole number line
```

PROFESSOR LOGIC

Mathematicians often use numbers instead of words because numbers are easier to calculate with. ON becomes 1. OFF becomes 0. RED becomes 0. YELLOW becomes 0.5. GREEN becomes 1. The meaning depends on the context, but the structure is the same.

I used 0.5 for yellow. Why? Because yellow is in the middle: not fully stopped, not fully going. Halfway. And 0.5 is halfway between 0 and 1. That is not a coincidence. The numbers are chosen to reflect the structure of the options.

ASK YOURSELF

Before reading on: what is the carrier set for each of the following? Write them out using curly braces.

1. A dice. 2. A yes or no question. 3. The grade on an exam (A, B, C, D, F). 4. A compass heading (rough: North, South, East, West).

Designated values

PROFESSOR LOGIC

Here is a subtlety that matters a great deal later. In most carrier sets, some values count as "true" or "holding" and others do not. In a logic where the carrier is {0, 1}, the value 1 is the designated value: it represents true, correct, happening. The value 0 is not designated.

This might seem obvious for a two-value system. It becomes interesting when you have more values. In the traffic light carrier {0, 0.5, 1}, which values are designated? It depends on what question you are asking. If the question is "should I go?", only 1 is designated. If the question is "should I be alert?", perhaps 0.5 and 1 are both designated.

The designation rule is a choice you make. Different choices give different systems.

Designated value: a value in the carrier that counts as "true" or "holding." In classical logic, the carrier is {0, 1} and the single designated value is 1. In a three-valued carrier, there may be multiple designated values.

ASK YOURSELF

In the carrier {0, 0.5, 1} with the designation rule "any value above zero is designated":

Which values are designated? Which are not? How many designated values are there?

CHAPTER 1 EXERCISES

★ **1.1** Write the carrier set for each situation. Justify your encoding choice (why those numbers?).

(a) A light with three settings: off, dim, bright. (b) A sensor reading soil moisture as a percentage. (c) A database field that can hold yes, no, or "not yet recorded".

Answer:

(a) {0, 0.5, 1}: off=0, dim=0.5, bright=1. Three evenly spaced values, reflecting equal steps in brightness. (b) [0, 1] or {0, 0.01, 0.02, ..., 1.0}: continuous range representing percentages. Best modelled as the interval [0,1] to allow any percentage. (c) {0, N, 1} where 0=no, 1=yes, N=not recorded. N is a third value that is neither yes nor no.

★ **1.2** A student argues: "All carrier sets can be written using only 0 and 1." Are they right? Give an example that supports their view and an example that defeats it.

Answer:

They are right for two-valued systems: any two options can be encoded as 0 and 1. They are wrong for continuous or many-valued systems: the carrier [0,1] contains infinitely many values between 0 and 1 that cannot all be encoded as just 0 and 1. The continuous number line cannot be captured with only two values.

★★ **1.3** Design a carrier for a court verdict system that needs to track: guilty, not guilty, and "verdict still pending." State your carrier, your encoding, and your designation rule. Explain what it would mean to change the designation rule.

Answer:

Carrier: {0, 0.5, 1} where 0=not guilty, 0.5=pending, 1=guilty. Designation rule: only 1 (guilty) is designated, representing cases where action is required. Alternative designation rule: {0.5, 1} both designated, representing cases that are not yet resolved as "not guilty." Changing the designation rule changes which values trigger consequences. The carrier itself (the raw options) stays the same.

★★★ **1.4** Can a carrier have zero elements? Can it have exactly one element? What happens to logic in each case?

Answer:

Zero elements: impossible as a logical carrier. With no values, no statement can be evaluated as anything. The system cannot say anything at all. One element: possible, but produces trivial logic. If V = {1}, then every statement evaluates to 1

(true). Nothing can be false. Every operation must return 1. This is called trivial logic, and it is useless for reasoning about anything that could be false.

CHAPTER 2

The Three Operations

The tools you use to work with any carrier.

PROFESSOR LOGIC

A carrier set by itself does not do anything. It just sits there, holding its options. To reason with values, you need operations: rules for taking one or more values and producing a new value.

Three operations appear in essentially every logic system ever built. They are called NOT, AND, and OR. I want to introduce them in a way that makes their formulas feel inevitable rather than arbitrary. Because they are inevitable. Once you choose your carrier, the formulas for these operations are forced.

NOT: the flip machine

PROFESSOR LOGIC

NOT takes one value and gives back its opposite. In the light switch carrier: NOT(ON) = OFF, NOT(OFF) = ON. NOT is a reversal machine. You put something in, you get the other thing out.

Using numbers, NOT is a formula. Let me write it.

CHALKBOARD

NOT

```
NOT(v) = 1 − v

Check:
NOT(1) = 1 − 1 = 0   ✓   (true becomes false)
NOT(0) = 1 − 0 = 1   ✓   (false becomes true)

In the three-value carrier {0, 0.5, 1}:
NOT(0.5) = 1 − 0.5 = 0.5   (the middle value is its own opposite)
```

> **Notice:** *The value 0.5 flips to itself under NOT. Nothing in the formula prevents this. Some values are their own opposite. This will become very important in Chapter 6.*

ASK YOURSELF

Compute NOT(NOT(v)) for v = 0, v = 0.5, and v = 1. What pattern do you see? Write it as a general rule before reading on.

PROFESSOR LOGIC

Did you get it? Flipping twice returns you to where you started. NOT(NOT(v)) = v. Always. This is called Double Negation, and we will prove it holds for every value in Chapter 3. For now, just notice that it is a consequence of the formula 1 minus v, not a rule we added separately.

AND: the strict gatekeeper

PROFESSOR LOGIC

AND takes two values and combines them. The result is designated only when both inputs are designated. Imagine a security door that requires both a keycard AND a PIN. One alone is not enough. Both, or nothing.

Here is the formula.

CHALKBOARD

AND

```
AND(a, b) = a × b    (multiplication)

Truth table for {0, 1}:
AND(0, 0) = 0 × 0 = 0
AND(0, 1) = 0 × 1 = 0
AND(1, 0) = 1 × 0 = 0
AND(1, 1) = 1 × 1 = 1   ← only this row gives 1

Why multiplication? Because the only way to get 1 × 1 = 1 is when both are 1.
Any zero in the input gives a zero output. The strict gatekeeper.
```

OR: the generous one

CHALKBOARD

OR

```
OR(a, b) = max(a, b)    (the larger of the two)

Truth table for {0, 1}:
OR(0, 0) = max(0, 0) = 0
OR(0, 1) = max(0, 1) = 1
OR(1, 0) = max(1, 0) = 1
OR(1, 1) = max(1, 1) = 1   ← fails only when both are 0

Why maximum? The designated value (1) is the largest.
As long as one input is 1, the maximum is 1. OR succeeds if either input
succeeds.
```

PROFESSOR LOGIC

Here is something worth pausing on. The formulas for AND and OR are not arbitrary choices. They follow from what AND and OR are supposed to mean, combined with the decision to use 0 and 1 as the values. Given those two decisions, the formulas are the natural result. Multiplication gives "both must be true." Maximum gives "at least one must be true." You did not need to memorise them. You could derive them.

Load: operations have costs

PROFESSOR LOGIC

Every operation has a cost. We will explore this fully in Chapter 5, but I want to plant the seed now. NOT is cheap: you just flip the value. AND is expensive: it adds the costs of both inputs together. OR is clever: it routes through the cheaper input.

This will matter when we encounter paradoxes. For now: remember that operations are not free.

ASK YOURSELF

Compute the following. Show your working.

1. NOT(NOT(0.5)) 2. AND(0.5, NOT(0.5)) 3. OR(0.5, NOT(0.5)) 4. AND(NOT(1), NOT(0))

CHAPTER 2 EXERCISES

★ **2.1** Build a complete truth table for NOT, AND, and OR on the carrier {0, 1}. Four rows for AND and OR (both inputs vary). Two rows for NOT (one input varies).

Answer:

NOT: NOT(0)=1, NOT(1)=0. AND: (0,0)=0, (0,1)=0, (1,0)=0, (1,1)=1. OR: (0,0)=0, (0,1)=1, (1,0)=1, (1,1)=1.

★ **2.2** Compute in the carrier {0, 0.5, 1} using NOT(v)=1−v, AND=multiply, OR=max:
(a) NOT(0.5) (b) AND(0.5, 0.5) (c) OR(0.5, 0.5) (d) NOT(AND(1, 0.5))

Answer:

(a) 0.5 (b) 0.25 (c) 0.5 (d) NOT(0.5) = 0.5

★★ **2.3** Implication (written A → B, meaning "if A then B") has this truth table in {0,1}: it equals 0 only when A=1 and B=0; in all other cases it equals 1. Show that A → B = OR(NOT(A), B). Test all four input combinations.

Answer:

OR(NOT(A), B): (A=0,B=0): OR(1,0)=1 ✓ (A=0,B=1): OR(1,1)=1 ✓ (A=1,B=0): OR(0,0)=0 ✓ (A=1,B=1): OR(0,1)=1 ✓. All four match. Implication is derivable from NOT and OR.

★★ **2.4** Why does OR use maximum rather than addition? (Hint: what would OR(1, 1) = 1 + 1 = 2 mean in a {0, 1} carrier? Is 2 in the carrier?)

Answer:

Addition would give OR(1,1) = 2, which is outside the carrier {0,1}. An operation must map carrier values to carrier values (closure). Addition violates closure for OR on {0,1}. Maximum is the correct formula because it always stays within the carrier.

★★★ **2.5** Design an operation called WEAK-AND for carrier [0,1] such that: WEAK-AND(1,1)=1, WEAK-AND(0,x)=0 for all x, and WEAK-AND(a,b) ≥ a×b for all a,b in [0,1]. Give a formula and prove the third property holds.

Answer:

Example: WEAK-AND(a,b) = min(a,b). Check WEAK-AND(1,1)=1 ✓. WEAK-AND(0,x)=min(0,x)=0 ✓. WEAK-AND(a,b)=min(a,b) ≥ a×b since for a,b in [0,1], min(a,b) ≥ a×b (if a≤b then min=a≥a×b since b≤1 means a×b≤a). ✓

CHAPTER 3

Laws That Cannot Break

Some truths are forced. Others are merely true for now.

PROFESSOR LOGIC

Here is a question I want you to sit with for a moment.

Is the law "a thing cannot be both true and false at the same time" something someone chose? A rule a philosopher decided to include? Or is it something that falls out of the arithmetic of {0, 1}?

The answer is the second one. And that distinction matters enormously. Because a rule that someone chose could be different if they had chosen otherwise. But a consequence of arithmetic cannot be different. It is forced.

CHALKBOARD

Forced law

A statement is a *forced law* of a carrier if it is true for every possible value in that carrier.

How to check: test every value. One failure is enough to disprove. No failures across all values: the law is forced.

The Law of Non-Contradiction

CHALKBOARD

LNC: AND(v, NOT(v)) = 0 for all v in V

Test in {0, 1}:
v = 0: NOT(0) = 1. AND(0, 1) = 0 × 1 = 0. ✓
v = 1: NOT(1) = 0. AND(1, 0) = 1 × 0 = 0. ✓

Both values checked. Both passed. LNC is *forced* in {0, 1}.

Now test in {0, 0.5, 1}:
v = 0: NOT(0) = 1. AND(0, 1) = 0. ✓
v = 1: NOT(1) = 0. AND(1, 0) = 0. ✓
v = 0.5: NOT(0.5) = 0.5. AND(0.5, 0.5) = 0.25. ✗ **0.25 ≠ 0**

One failure. LNC is *not forced* in {0, 0.5, 1}.

> **PROFESSOR LOGIC**
>
> Same formula. Same operations. Different carrier. Different result. The law did not change. The arithmetic changed. This is what it means to say the laws are forced by the carrier, not chosen by us.

The Law of Excluded Middle

CHALKBOARD

LEM: OR(v, NOT(v)) = max(V) for all v in V

(In {0,1}: max(V) = 1. In {0, 0.5, 1}: max(V) = 1.)

Test in {0, 1}:
v = 0: OR(0, 1) = 1. ✓
v = 1: OR(1, 0) = 1. ✓
LEM is forced in {0, 1}.

Test in {0, 0.5, 1}:
v = 0.5: NOT(0.5)=0.5. OR(0.5, 0.5) = 0.5. 0.5 ≠ 1. ✗
LEM is *not forced* in {0, 0.5, 1}.

Double Negation

CHALKBOARD

DN: NOT(NOT(v)) = v for all v in V

For NOT(v) = 1 − v:

NOT(NOT(v)) = 1 − (1 − v) = v ✓ always

This is pure algebra. It holds for any value in any carrier that uses NOT(v) = 1−v.

It does not require checking case by case. The formula proves it directly.

The master table

Law	{0, 1} Classical	{0, 0.5, 1} Paraconsistent	[0, 1] Fuzzy
Non-Contradiction	✓ forced	✗ fails at 0.5	✗ fails everywhere except 0 and 1
Excluded Middle	✓ forced	✗ fails at 0.5	✗ fails everywhere except 0 and 1
Double Negation	✓ forced	✓ forced	✓ forced

Notice: *Double Negation is the most robust law. It survives carrier changes that destroy LNC and LEM. This is because it depends only on the formula for NOT, not on AND or OR.*

"The laws are not carved in stone. They are printed on carrier paper. Change the carrier and different laws come with it."

CHAPTER 3 EXERCISES

★ **3.1** Check whether LNC holds in the carrier {0, 0.33, 0.67, 1} using AND=multiply and NOT(v)=1−v. Test every value. State clearly which values pass and which fail.

Answer:

v=0: AND(0,1)=0 ✓. v=0.33: NOT(0.33)=0.67, AND(0.33,0.67)=0.33×0.67=0.221 ≠0 ✗. v=0.67: AND(0.67,0.33)=0.221 ≠0 ✗. v=1: AND(1,0)=0 ✓. LNC fails at 0.33 and 0.67.

★★ **3.2** A student claims: "If LNC fails in a carrier, LEM must also fail." Test this claim. Find a carrier where LNC fails but LEM holds, or prove no such carrier can exist.

Answer:

Constructive example: use V = {0, 0.5, 1} with AND = min instead of multiply. LNC at 0.5: AND(0.5, NOT(0.5)) = min(0.5, 0.5) = 0.5 ≠ 0. LNC fails. LEM at 0.5: OR(0.5, NOT(0.5)) = max(0.5, 0.5) = 0.5 ≠ 1. LEM also fails. Try an asymmetric NOT: let NOT(0.5) = 0. Then LNC at 0.5: AND(0.5, 0) = 0 ✓. LNC holds. LEM at 0.5: OR(0.5, 0) = 0.5 ≠ 1. LEM fails. So LNC holding does not force LEM. The student's claim (LNC failure implies LEM failure) remains unrefuted by this example, but the converse is false.

★★ **3.3** Prove that Double Negation holds for ANY carrier, as long as NOT(v) = 1−v. (Hint: use algebra rather than testing individual values.)

Answer:

NOT(NOT(v)) = NOT(1−v) = 1−(1−v) = 1−1+v = v. This algebraic identity holds for any real number v, so it holds for any carrier whose values are real numbers and whose NOT uses the formula 1−v. QED.

★★★ **3.4** Is it possible to design a carrier where LNC holds but Double Negation fails? If yes, construct one. If no, explain why not.

Answer:

Yes. Define NOT differently. Let V = {0, 1} and NOT(0) = 0, NOT(1) = 0. Then NOT(NOT(0)) = NOT(0) = 0 ✓, but NOT(NOT(1)) = NOT(0) = 0 ≠ 1. DN fails at v=1. Check LNC: AND(0, NOT(0)) = AND(0,0) = 0 ✓. AND(1, NOT(1)) = AND(1, 0) = 0 ✓. LNC holds. So yes: LNC can hold while DN fails, if we use a non-standard NOT.

CHAPTER 4

Three Knobs

Every formal system is completely specified by exactly three things.

PROFESSOR LOGIC

Here is something that will rearrange how you think about logic, mathematics, and formal systems of all kinds.

Every formal system ever built is completely specified by three parameters. Three knobs on a machine. Turn one knob and some laws change. The others stay. Turn a different knob and a different set of laws changes. The mechanism never changes. Only the outputs do.

Let me show you the three knobs.

CHALKBOARD

The three parameters

```
V  — the value carrier   (what values are possible)
Γ  — the gradient family   (which operations are available)
θ  — the coherence threshold   (how much load the system tolerates)

Change V: different laws become forced.
Change Γ: some laws become unreachable.
Change θ: different patterns can cohere.
```

Knob 1: the value carrier (V)

PROFESSOR LOGIC

We have already seen this one in action. Changing V from {0, 1} to {0, 0.5, 1} broke LNC and LEM but left Double Negation intact. The operations were identical. Only the carrier changed.

Here is the full picture of what changing V does.

CHALKBOARD

```
Classical:   V = {0, 1}   →   LNC forced, LEM forced, Liar is paradox
Three-valued:   V = {0, 0.5, 1}   →   LNC fails, LEM fails, Liar resolves at
0.5
Fuzzy:   V = [0, 1]   →   LNC fails everywhere, Liar resolves at 0.5
Calculus:   V = ℝ   →   Different laws entirely (product rule, chain rule)
Probability:   V = [0, 1] with normalisation   →   Kolmogorov's axioms
```

Knob 2: the gradient family (Γ)

PROFESSOR LOGIC

The gradient family is the set of operations available in the system. We have been working with {NOT, AND, OR}. But you can restrict this. And when you do, some laws become unreachable: not false, just impossible to state.

The clearest example: intuitionistic logic. Same carrier as classical logic, V = {0, 1}. But OR is removed from the gradient family. What happens?

CHALKBOARD

```
Classical logic:   Γ = {NOT, AND, OR, →}
Intuitionistic logic:   Γ = {NOT, AND, →}   (OR removed)

LEM says:   OR(v, NOT(v)) = 1

Without OR in Γ, this statement cannot even be formed.
LEM is not false in intuitionistic logic. It is unreachable.

LNC still holds (uses AND, which is still in Γ).
The system is not "broken." It is more restricted. More careful.
```

PROFESSOR LOGIC

Intuitionistic logic is used in computer science for constructive proofs: you can only say something is true if you can construct a specific proof of it, not just show that assuming it is false leads to a contradiction. Removing OR removes the ability to assert "either this or its opposite must hold" without producing the actual witness.

Knob 3: the coherence threshold (θ)

PROFESSOR LOGIC

Every time you apply an operation, the result carries some cost. A pattern with a cost above θ cannot cohere in the system. We will explore this fully in Chapter 5. For now: θ is the third knob, and it controls how complex a pattern can be before the system can no longer hold it.

Think of it as a circuit breaker. Too much load, and the circuit opens. The system shuts down gracefully rather than producing nonsense.

ASK YOURSELF

Without doing any calculation: if I lower θ from 1.0 to 0.5, what do you predict will happen? Will there be more or fewer patterns that can cohere? Will the system become more or less powerful?

CHAPTER 4 EXERCISES

★ **4.1** For each change below, state which laws change and which stay the same. Do not calculate: reason from what you know about each knob.

(a) Change V from {0,1} to {0, 0.5, 1}. (b) Remove OR from Γ. (c) Double θ from 1.0 to 2.0.

Answer:

(a) LNC and LEM break. DN survives. (b) LEM becomes unreachable (not false, unreachable). LNC still holds. Laws that use AND or NOT are unaffected. (c) More patterns can cohere. No logical laws change (they are about values, not costs). The system becomes more permissive about complex patterns.

★★ **4.2** Linear logic adds a consumption constraint to AND: each pattern can only be used once. This is a change to Γ. What real-world situation does this model? Name two fields where this constraint is natural.

Answer:

Linear logic models resource consumption: using something means it is no longer available. Fields: (1) Economics and resource allocation (spending money depletes it). (2) Computer memory management (a pointer used in one place cannot be used elsewhere without copying). Also: quantum computing (the no-cloning theorem), chemistry (reactants consumed in a reaction).

★★★ **4.3** Can you change all three parameters simultaneously and still recognise the result as a "logic"? What is the minimum requirement for something to deserve the name?

Answer:

Yes. You can change all three. The minimum requirements are: (1) V is non-empty (at least one value). (2) The operations in Γ are closed (they only produce values in V). (3) θ is a positive real number. As long as these hold, you have a consistent formal system. Whether it is useful is a separate question. Very unusual choices of V and Γ might produce trivial systems (everything true, or nothing true), but they are still formally valid.

CHAPTER 5

The Price of an Idea

Every distinction costs something. The universe insists on this.

PROFESSOR LOGIC

In 1961, a physicist named Rolf Landauer published a paper arguing that erasing one bit of information requires a minimum amount of energy. Not as a practical matter, but as a matter of physics. The thermodynamic cost of destroying a distinction between two states cannot be zero.

At room temperature, erasing one bit costs at least:

CHALKBOARD

$$L_{min} = k_B \times T \times \ln(2) \approx 2.87 \times 10^{-21} \text{ joules}$$

k_B = Boltzmann's constant (1.38×10^{-23} J/K)
T = temperature in Kelvin (room temperature ≈ 300K)
ln(2) ≈ 0.693

This is called the Landauer limit. It is a physical floor, not an engineering limitation.

PROFESSOR LOGIC

Why does this matter for logic?

Every logical operation is, at its core, a manipulation of distinctions. True and false are distinct. Maintaining, combining, and erasing these distinctions requires work. The universe charges for this service. We can abstract the cost as a dimensionless number called *load*, and we track it through every operation.

Load combination rules

CHALKBOARD

A pattern is written as P = (value, load)

NOT(P): value = $1 - v$ load = L *(unchanged: just a flip)*

AND(P, Q): value = $v_P \times v_Q$ load = $L_P + L_Q$ *(loads add: expensive)*

OR(P, Q): value = $\max(v_P, v_Q)$ load = $\min(L_P, L_Q)$ *(takes cheaper path)*

PROFESSOR LOGIC

AND is expensive because it requires both patterns to hold simultaneously. The system must maintain both distinctions at once. Their costs add. OR is clever: it only needs one of the two patterns, so it routes through the cheaper one. The costly one can be ignored.

This is why OR tends to be cheaper than AND in any system that tracks costs. And it explains something very intuitive: "A is true OR B is true" is easier to establish than "A is true AND B is true."

The coherence threshold

CHALKBOARD

θ = maximum load a pattern can carry and still cohere.

If load(P) ≤ θ → P is coherent. It can be used in further reasoning.
If load(P) > θ → P is incoherent. The system cannot hold it.

Example: θ = 10.
AND((1, 4), (1, 3)) = (1, 7) ✓ **coherent**
AND((1, 7), (1, 5)) = (1, 12) ✗ **incoherent (12 > 10)**

DRAS: carrying your history

PROFESSOR LOGIC

Here is a subtle but important idea. The fine structure constant of physics is approximately 1/137 at low energies and 1/129 at high energies. It changes with scale. It is not constant at all.

The De-Reification Axiom Standard (DRAS) applies this insight to formal systems. Every quantity should carry its full history, including the scale at which it was measured. Writing just "1/137" without specifying the energy scale is a reification: treating a context-dependent measurement as if it were context-free.

In formal logic: every pattern P = (value, load) is the DRAS version of a logical statement. The value is what you would normally write. The load is the hidden cost you would normally ignore. Carrying both is more honest.

ASK YOURSELF

A proof has seven AND operations in sequence. Each input pattern has a load of 2.0. What is the load after three AND operations? After seven? If $\theta = 25$, does the proof cohere? At which step does it first fail?

CHAPTER 5 EXERCISES

★ **5.1** Compute the value and load of each result. θ = 10. State whether each is coherent.

(a) NOT((1, 4.0)) (b) AND((1, 3.0), (0, 5.0)) (c) OR((1, 8.0), (1, 3.0))

Answer:

(a) NOT: value=0, load=4.0. Coherent (4≤10). (b) AND: value=0, load=8.0. Coherent (8≤10). (c) OR: value=1, load=min(8,3)=3.0. Coherent (3≤10).

★ **5.2** Why is OR cheaper than AND? Explain in plain language, then connect your explanation to the load formulas.

Answer:

OR needs only one of its inputs to be true, so it takes the minimum of the two loads: the "cheaper path." AND needs both inputs simultaneously, so both costs must be paid: the loads add. In plain language: "either this OR that" is easier to maintain than "this AND that" because you only need to keep one of them active.

★★ **5.3** Modus Ponens: from P → Q and P, conclude Q. Using load rules, if P = (1, 3.0) and Q = (1, 7.0), what is the load of the conclusion? Is the load of the conclusion larger or smaller than the load of P alone?

Answer:

P → Q = OR(NOT(P), Q) = OR((0, 3.0), (1, 7.0)) = (1, min(3,7)) = (1, 3.0). Applying the implication to P: AND((1,3), (1,3)) = (1, 6.0). So the conclusion carries load 6.0, which is larger than P alone (3.0). Reasoning has costs even when the conclusion is certain.

★★★ **5.4** A student says: "If we set θ = ∞, every proof will cohere, so we will get a complete logic." Is this correct? What does Gödel's incompleteness theorem say about this?

Answer:

The student is partially right: with θ = ∞, no pattern is ever rejected for load reasons. But Gödel's theorem says that for any sufficiently powerful system, some true statements require self-referential proofs whose load grows without bound (2^depth). Even at θ = ∞, you would need an infinite chain of reasoning steps for some statements. You cannot complete those proofs in a finite time. The problem is not the threshold; it is the infinite load itself. DRAS reframes Gödel as a thermodynamic observation: some truths cost infinite work to establish.

CHAPTER 6

The Debt of the Distinction

Every boundary costs something. Ignore that cost long enough and the universe sends a bill.

PROFESSOR LOGIC

In the last chapter we met Landauer's principle: erasing a single bit of information costs at minimum 2.87×10^{-21} joules at room temperature. We used that as a physical grounding for load.

But there is a deeper consequence that I want to sit with now. Not just the cost of erasing a distinction. The cost of *maintaining* one.

A boundary between "true" and "false" is not a line drawn in ink. It is an active physical process. Something must hold those two states apart. Something must prevent the blurring. In biology, a cell membrane maintains the distinction between inside and outside at continuous thermodynamic cost. In computing, a flip-flop maintains the distinction between 0 and 1 at continuous power consumption. In language, a culture maintains the distinction between "heap" and "not heap" at continuous cognitive cost.

Classical logic does not charge for this maintenance. It treats all distinctions as free. That is a debt, not a free lunch. And the universe collects eventually.

What a boundary actually is

CHALKBOARD

```
In classical logic:

    TRUE      |      FALSE
               ^
               |
               This line is assumed to be
               infinitely sharp and free to maintain.

In reality:

    TRUE    [gradient zone]    FALSE
              ^
              This region costs energy to resolve.
              The closer a state is to the boundary,
              the more it costs to classify.
```

PROFESSOR LOGIC

Consider a medical test that returns POSITIVE or NEGATIVE. The test is measuring a continuous biological quantity: a hormone level, a protein concentration, an electrical signal. At some threshold, the clinician has drawn a line and said "above this is positive, below this is negative."

That line is a classical {0, 1} distinction imposed on a [0, 1] physical reality. The distinction is useful. But it is not free. Every case near the threshold pays the debt in a concrete form: false positives and false negatives. The logic said "this is a crisp boundary." The biology did not get the memo.

CHALKBOARD: TYPES OF DISTINCTION DEBT

Precision debt:
Claiming more certainty than the system actually has.
The logic returns TRUE or FALSE. The measurement was 0.51.
Cost: false confidence. Errors that look like correct results.

Boundary debt:
Maintaining a sharp line where nature has a gradient.
Cost: edge cases. Exceptions. The category "other." The asterisk in every policy.

Accumulation debt:
Each AND operation adds the loads of its inputs.
Classical logic does not track this. But the system does.
Cost: a chain of reasoning that seems valid but rests on distinctions whose combined maintenance cost exceeds what the context can sustain.

The pragmatic bargain

PROFESSOR LOGIC

I want to be clear about something important. Classical logic is not wrong. It is a pragmatic bargain. You agree to treat all distinctions as free and sharp, and in return you get a powerful, tractable formal system that is useful for an enormous range of problems.

The bargain is worth making in many contexts. When your problem is genuinely binary, the cost of the distinction is small and the power of the system is large. The debt is real but manageable.

The problem is when the bargain is made without acknowledgement. When a binary logic is applied to a continuous problem and the hidden costs are called "edge cases" or "exceptions" rather than "the distinction debt we forgot to account for." The debt does not disappear when it is unacknowledged. It compounds.

CHALKBOARD: WHEN THE DEBT IS CALLED IN

```
The debt of the distinction appears as:

  — The exception that breaks the rule
  — The edge case that crashes the software
  — The policy that is "technically correct" but produces absurd outcomes
  — The experiment whose results "don't quite fit" the model
  — The medical misdiagnosis of a borderline case
  — The legal verdict that everyone knows is technically right but morally
wrong

All of these are the same structural event: a distinction was maintained
past the point where the underlying reality supported it.
```

PROFESSOR LOGIC

DRAS, the De-Reification Axiom Standard, is the honest accounting. Instead of treating every quantity as a bare constant with a crisp value, DRAS requires you to carry its full history: the load it has accumulated, the context at which it was measured, the scale at which the distinction was drawn.

A medical threshold is not "the value is 0.5 per millilitre." It is "at this laboratory, with this instrument, in this population, the threshold that minimises the combined cost of false positives and false negatives was found to be 0.5 per millilitre." The full DRAS form carries all of that. The reified form strips it away and pretends the number is context-free.

When the context changes and the number is not, the debt is called in.

How to price the debt honestly

CHALKBOARD

For any classical distinction you are making:

1. Ask: what is the actual value space of the underlying quantity?
   If it is continuous: you are drawing a line in a gradient.

2. Ask: how far from the boundary are most real cases?
   If they cluster near the boundary: the debt is high.
   If they cluster away from the boundary: the debt is low.

3. Ask: what happens to cases at the boundary?
   If they are classified arbitrarily: that cost is the unpaid debt.

4. Ask: what would change if you extended V to include the boundary as a value?
   Usually: fewer exceptions, more complexity, more honest results.

The answer to question 4 tells you whether the debt is worth paying.

> **Notice:** *The distinction debt is not an argument against using classical logic. It is an argument for being honest about what you are doing when you use it. A two-value system is a model. All models simplify. The question is always: what does this model cost, and is that cost worth paying?*

CHAPTER 6 EXERCISES

★ **6.1** A speed camera photographs a car. The software classifies it as SPEEDING or NOT SPEEDING using a threshold of 30 mph. The measurement uncertainty is ±2 mph. A car measured at 31 mph is classified as SPEEDING. Identify the distinction debt in this scenario. What would DRAS require?

Answer:

The distinction debt: the measurement has uncertainty of ±2 mph, meaning the true speed is somewhere in [29, 33] mph. The sharp threshold at 30 mph cannot distinguish a car genuinely doing 31 mph from one doing 29 mph (within measurement error). DRAS would require: carry the measurement as (31 mph, ±2 mph uncertainty, 95% confidence). The threshold decision should reflect the uncertainty: a car at 31±2 is borderline and the decision carries a 50% probability of error.

★★ **6.2** A hiring algorithm uses 50 binary features to classify applicants as HIRE or REJECT. Each feature is 90% accurate. What is the probability that a single decision is correct, assuming independence? What is the distinction debt accumulating across all 50 features?

Answer:

Probability all 50 features correct: 0.9^50 = 0.0052. About 0.5% chance the full decision is "correct" by this measure. Each AND of two features multiplies accuracy: 0.9 × 0.9 = 0.81. The distinction debt is the gap between the claimed precision (a clean HIRE/REJECT) and the actual uncertainty (0.52% reliability). The system produces a confident binary output for decisions that are essentially noise.

★★ **6.3** Describe a situation in your own area of study or experience where a classical {0,1} distinction is being applied to a genuinely continuous phenomenon. Estimate whether the debt is being acknowledged or hidden.

Answer:

Open question. Look for: binary grades (A/F) on continuous performance. Pass/fail exams on continuous understanding. Legal guilty/not guilty on continuous evidence. Poverty lines on continuous income. Any binary policy on continuous human need.

★★★ **6.4** Design a "distinction cost function" for the carrier {0, 1} using some external knowledge about a domain of your choice. The function D(x) should return the cost of maintaining the boundary at point x in the underlying continuous space.

Sketch how D(x) might look for a medical test, a speed camera, and a literary genre classification ("is this book science fiction?").

Answer:

Medical test: D(x) is a U-shaped curve centred at the threshold. Very low near 0 and 1 (clear cases). Very high near the boundary. Formally: D(x) = 1 / |x - threshold|. Speed camera: D(x) is approximately flat but spikes sharply at the speed limit. Literary genre: D(x) has multiple spikes (many genre boundaries are contested) and is generally much higher than physical measurements, reflecting that genre is a social distinction maintained by negotiation, not measurement.

CHAPTER 7

When Systems Break

Four modes. Each one a different kind of overload.

CHAPTER 6

When Systems Break

Four modes. Each one a different kind of overload.

PROFESSOR LOGIC

A paradox is not a mystery waiting for a philosophical answer. A paradox is a gradient overload. The system demanded a cost it could not pay.

There are exactly four ways this can happen. I want to show you all four, and I want to show you that each one is a distinct failure mode with a distinct structure. Two thousand years of philosophy has been, in part, a conversation about these four modes without having the vocabulary to distinguish them.

Mode 1: The Liar (no fixed point)

CHALKBOARD

"This sentence is false."

Let v = truth value of the sentence.
The sentence demands: v = NOT(v)

Test in {0, 1}:
v = 0: NOT(0) = 1 ≠ 0. ✗
v = 1: NOT(1) = 0 ≠ 1. ✗

No value satisfies v = NOT(v). The equation has no solution in {0,1}.
The pattern demands a value the carrier does not contain.

In {0, 0.5, 1}:
v = 0.5: NOT(0.5) = 0.5 = v. ✓ **Resolved.**

The Liar is not a mystery. It is the carrier {0,1} not containing 0.5.

PROFESSOR LOGIC

The Liar paradox is two thousand years old. Epimenides of Crete, around 600 BCE, said "all Cretans are liars" (he was Cretan). Philosophers have argued about it ever since. The argument dissolves the moment you look at it as an equation in a value space. The equation v = NOT(v) needs v = 0.5 as a solution. If the value space does not contain 0.5, there is no solution. Not "no philosophical answer." No arithmetic solution. Different thing entirely.

Mode 2: Load diverges (Gödel, Turing)

CHALKBOARD

Mode 2: the load grows without bound

A self-referential proof requires its own proof as a premise.
Depth 1: load = 1
Depth 2: load = 2 (references depth 1)
Depth 3: load = 4 (references depth 2)
Depth n: load = 2^n

For any finite θ, there exists a depth n where $2^n > \theta$.
The proof cannot cohere. Not because it is false. Because it costs too much.

Gödel's incompleteness: every sufficiently powerful system contains true statements
that are too expensive to prove within the system. This is the Mode 2 explanation.

Mode 3: Level collapse (Russell)

PROFESSOR LOGIC

Bertrand Russell, in 1901, discovered that naive set theory allows the set R = {x : x is not a member of itself}. Ask whether R is a member of itself and the same flip-flop appears. But this is structurally different from the Liar. There is no infinite load accumulation. Instead, the gradient family demands a context one level larger than the one available to evaluate R's membership. The system needs to look at itself from outside itself. It cannot.

CHALKBOARD

Mode 3: the gradient demands a larger context

Russell's set R: is R a member of R?
If yes → R should not be a member (by definition). ✗
If no → R is exactly the kind of set that belongs in R. ✗

Resolution: type theory. Separate the levels.
Sets of objects. Sets of sets. Sets of sets of sets.
Each level can only refer to levels below it.
Higher axiom cost. More careful bookkeeping. No paradox.

Mode 4: No seed (Yablo)

PROFESSOR LOGIC

In 1993, Stephen Yablo constructed a paradox with no self-reference at all. A sequence of sentences: sentence 1 says "all sentences after me are false." Sentence 2 says the same about everything after sentence 2. And so on, forever. To evaluate sentence 1 you need sentence 2. To evaluate sentence 2 you need sentence 3. There is no starting point. No base case. No seed.

This is fundamentally different from all three previous modes. No flip-flop. No infinite load accumulation. No level mismatch. Just a structure with no beginning.

CHALKBOARD

Mode 4: no seed state

S_1: "All S_k for k > 1 are false."
S_2: "All S_k for k > 2 are false."
S_3: "All S_k for k > 3 are false."
...

To evaluate S_1 you need S_2. To evaluate S_2 you need S_3. No starting point.
The construction has no base case. It cannot be seeded.

No self-reference. Mode 4 is structurally distinct from Mode 1.

The four modes at a glance

Mode	Name	Core failure	Classic example	Resolution
1	Liar	NOT has no fixed point in V	This sentence is false	Extend V
2	Gödel	Self-referential load diverges	Gödel sentence, halting problem	Cannot be fully resolved
3	Russell	Gradient demands larger context	Russell's paradox	Type stratification
4	Yablo	No seed state	Yablo's sequence	Cannot be fully resolved

CHAPTER 6 EXERCISES

★ **6.1** Solve v = NOT(v) in each carrier: (a) {0, 1}, (b) {0, 0.5, 1}, (c) {0, 0.25, 0.5, 0.75, 1}.

Answer:

(a) No solution: v=0.5 required but not in carrier. (b) Solution: v=0.5. (c) Solution: v=0.5 (present in this carrier). Note: {0, 0.25, 0.5, 0.75, 1} resolves the Liar for the same reason as (b).

★★ **6.2** Classify each as Mode 1, 2, 3, or 4. Justify your answer.

(a) "The next sentence is true. The previous sentence is false."

(b) The set of all ordinals (the Burali-Forti paradox).

(c) A program that prints itself and halts, if it would otherwise loop forever.

Answer:

(a) Mode 1: two sentences creating a mutual flip-flop. Each demands the NOT of the other. No fixed point. (b) Mode 3: the collection of all ordinals would itself be an ordinal, requiring a context larger than the one being defined. Level collapse. (c) Mode 2: a self-referential decision procedure. The halting problem requires a machine to simulate itself, generating infinite load recursion.

★★★ **6.3** Design a carrier that resolves Mode 1 (the Liar) but still exhibits Mode 2 (load divergence). Explain why resolving Mode 1 does not resolve Mode 2.

Answer:

Carrier {0, 0.5, 1}: Mode 1 resolved (v=0.5 is the fixed point). Mode 2 still holds because Mode 2 is about load growth, not about the value space. Self-referential load grows as 2^n regardless of whether 0.5 is available as a value. The Liar resolves to 0.5 in this carrier, but Gödel's construction depends on proving a statement about provability, not about a truth-value fixed point. The two modes are orthogonal.

CHAPTER 7

Build Your Own

Five steps. One formal system. Yours.

PROFESSOR LOGIC

Everything in the previous six chapters was analysis: I showed you formal systems that already exist, and we worked out their properties. Now it is time for synthesis. You are going to design a formal system from scratch.

This is the skill that transfers everywhere. Every branch of mathematics and computer science is, in part, a design problem: which value space, which operations, which threshold. When you know how to build one, you know how to read all of them.

CHALKBOARD

The five steps

```
1. Choose V — your value space
2. Define NOT, AND, OR — for every value in V
3. Set θ — your coherence threshold
4. Derive forced laws — test every value
5. Find failure modes — what breaks, and under what conditions
```

Worked example: the traffic light carrier

PROFESSOR LOGIC

Let me build one from scratch and show you the full process. I will use a traffic light, because we have already met it and the choices feel natural.

CHALKBOARD: STEP 1 — CHOOSE V

Traffic lights have three states: stop, caution, go.
Encode: RED = 0, YELLOW = 0.5, GREEN = 1

V = {0, 0.5, 1}

CHALKBOARD: STEP 2 — DEFINE OPERATIONS

NOT: the opposite instruction.
  NOT(RED) = GREEN. NOT(GREEN) = RED. NOT(YELLOW) = YELLOW.
  Formula: NOT(v) = 1 − v. *(This handles all three correctly.)*

AND: the more cautious of two signals. Use AND = min.
  AND(GREEN, YELLOW) = min(1, 0.5) = 0.5 (be cautious).

OR: the more permissive of two signals. Use OR = max.
  OR(RED, YELLOW) = max(0, 0.5) = 0.5 (proceed with caution).

CHALKBOARD: STEPS 3, 4, 5

Step 3: θ = 1.0 (crisp threshold)

Step 4 — Check LNC: AND(v, NOT(v)) = min(v, 1−v)
  v=0: min(0,1) = 0. ✓
  v=0.5: min(0.5,0.5) = 0.5. **✗ (0.5 ≠ 0)**
  v=1: min(1,0) = 0. ✓
LNC fails at YELLOW.

Check LEM: OR(v, NOT(v)) = max(v, 1−v)
  v=0.5: max(0.5, 0.5) = 0.5. **✗ (0.5 ≠ 1)**
LEM fails at YELLOW.

Step 5 — Failure mode: YELLOW AND NOT(YELLOW) = YELLOW.
Yellow is simultaneously "caution" and "caution's opposite."
That is accurate: a yellow light is genuinely both-and-neither.
The system is not broken. It is capturing the actual ambiguity correctly.

PROFESSOR LOGIC

This is an important point. When LNC fails in your carrier, that is not necessarily a problem. It depends on whether the failure corresponds to something real in your domain. For traffic lights, a yellow signal genuinely is in a state of tension between "stop required" and "proceed possible." The mathematics is being honest about the physics.

The design checklist below will help you keep track as you build.

CHALKBOARD: DESIGN CHECKLIST

Before you declare a carrier finished, check every box:

- ☐ Is V clearly defined with at least two values?
- ☐ Is NOT defined for every value in V?
- ☐ Is AND defined for every pair of values?
- ☐ Is OR defined for every pair of values?
- ☐ Does every operation produce a result that is in V? (Closure)
- ☐ Have you checked LNC for every value?
- ☐ Have you checked LEM for every value?
- ☐ Have you checked Double Negation for every value?
- ☐ Have you identified at least one failure mode?
- ☐ Can you state in one sentence what this logic is useful for?

CHAPTER 7 EXERCISES

★★ **7.1** Build a carrier for a blood-oxygen monitor with values {critical, low, normal, high} = {0, 0.3, 0.7, 1}. Define NOT for each value (what is the "opposite" of each level?). Use AND = min, OR = max. Check LNC, LEM, and Double Negation. Report at least one failure mode.

Answer:

Plausible NOT: NOT(0)=1, NOT(1)=0, NOT(0.3)=0.7, NOT(0.7)=0.3. Check LNC: AND(0.3, NOT(0.3))=AND(0.3,0.7)=min(0.3,0.7)=0.3≠0 — fails at low and high (by symmetry). LEM: OR(0.3, 0.7)=0.7≠1 — fails at low and high. Double Negation: NOT(NOT(0.3))=NOT(0.7)=0.3 ✓ — forced. Failure mode: AND(low, NOT(low))=low, meaning a reading can simultaneously be "low AND not-low" with value 0.3. This might be appropriate for a sensor with measurement uncertainty.

★★★ **7.2 (Open design)** Design a carrier for a domain of your choice. Document all five steps. State which laws hold and which fail, and explain whether the failures are problems or features for your domain.

Answer:

Open question. Any thoughtful answer that follows the five steps is correct. The key insight to look for: the student should be able to say whether a failure mode is "the logic is broken" or "the logic is capturing real ambiguity in the domain."

CHAPTER 8

One Mechanism, Many Faces

Calculus, probability, and modal logic are all the same system in different clothes.

PROFESSOR LOGIC

I want to show you something that surprises most people the first time they see it. And I want to be clear that it is not an analogy. Not "logic is kind of like calculus." I mean structurally identical. The same arithmetic. Different value spaces.

Calculus is logic with V = ℝ

CHALKBOARD

```
Set V = ℝ (all real numbers). Change the gradient family to differential
operations.

The load of a pattern P becomes its derivative:   load(P) = dP/dx

Now apply the AND load rule to two patterns P and Q:
load(AND(P, Q)) = load(P) × value(Q) + value(P) × load(Q)

This is identical to the product rule of calculus:
d/dx [f(x) × g(x)] = f'(x) × g(x) + f(x) × g'(x)

✓ The load rule for AND on V = ℝ IS the product rule. Not an analogy.

The Fundamental Theorem of Calculus (integration and differentiation are
inverses)
is the Double Negation of the calculus carrier. Same structure.
```

Probability is logic with normalisation

CHALKBOARD

```
Set V = [0, 1]. Interpret values as probabilities.
Add one constraint: probabilities of mutually exclusive events must sum to 1.

The forced laws of this carrier are Kolmogorov's axioms of probability.
P(A) ≥ 0 — values are non-negative (V = [0,1] enforces this)
P(certain event) = 1 — max(V) = 1
P(A or B) = P(A) + P(B) for mutually exclusive A, B — from normalisation

Shannon entropy (uncertainty of a distribution) = the load of that
distribution.
Maximum entropy = maximum load = most spread-out distribution.
```

Modal logic is logic with worlds

PROFESSOR LOGIC

Modal logic adds two operators: □ (necessarily) and ◊ (possibly). In carrier terms: the value space is extended to include multiple "worlds" (possible situations), and the gradient family gains an accessibility relation between worlds. Which worlds can "see" which other worlds determines which modal logic you get.

S4 logic: accessibility is reflexive (you can see yourself) and transitive (if you can see A and A can see B, you can see B). S5 logic: every world can see every other world. These are carrier choices. Different accessibility choices force different modal theorems.

> *"Classical logic, fuzzy logic, calculus, probability, and modal logic are all one operator — P / G → Q — applied to different value spaces. The diversity is in the carrier. The mechanism is singular."*

CHAPTER 8 EXERCISES

★ **8.1** In the calculus carrier (V = ℝ, load = derivative), what is the load of a constant function f(x) = 7? What does this tell you about constants in DRAS?

Answer:

The derivative of f(x)=7 is 0. Load = 0. Constants carry zero load: they have no dependence on the variable. In DRAS, a zero-load quantity is one that does not change with scale. A true constant is scale-independent. Any quantity with non-zero load varies with context.

★★ **8.2** Compute Shannon entropy $H = -\Sigma\, p_i \times \log_2(p_i)$ for: (a) a fair coin (p = 0.5, 0.5), (b) a biased coin (p = 0.9, 0.1), (c) a certain outcome (p = 1, 0). Which carries the most load?

Answer:

(a) H = −(0.5 log 0.5 + 0.5 log 0.5) = −2(0.5 × −1) = 1 bit. (b) H = −(0.9 log 0.9 + 0.1 log 0.1) ≈ 0.469 bits. (c) H = −(1 × 0 + 0 × log 0) = 0 bits. The fair coin carries the most load. Maximum uncertainty = maximum load.

★★★ **8.3** A sceptic says: "These are just analogies. The actual mathematics is completely different in each case." Write a 150-word response. What specific evidence would distinguish a genuine structural identity from a mere analogy?

Answer:

A mere analogy maps concepts to concepts loosely. A structural identity maps operations to operations precisely. The test: write down the load rule for AND in carrier-set terms. Write down the product rule in calculus terms. They are the same formula, with the same variables in the same positions. If they were merely analogous, the formulas would be different and the mapping would require interpretation at each step. Here, no interpretation is required. Load(AND(P,Q)) = Load(P)×Value(Q) + Value(P)×Load(Q). And d/dx[f×g] = f'g + fg'. Same arithmetic. Same structure. The evidence for structural identity: a single unified proof that derives both as consequences of the same mechanism on different value spaces. That proof exists and runs in the pl_unified.py reference implementation.

CHAPTER 9

Language as a Carrier

We have been using language to describe formal systems. Language is a formal system.

PROFESSOR LOGIC

We need to turn the framework back on itself.

For nine chapters I have been using language to describe carriers, operations, and forced laws. I have been using the subject-predicate structure of English to say things like "LNC is forced in {0,1}" and "the Liar Paradox has no solution in the classical carrier." Every sentence I have written is itself an operation on a value space.

What is the carrier of natural language? What are its operations? What are its forced laws? What is the debt we incur when we maintain linguistic distinctions?

These are not rhetorical questions. They have specific answers, and the answers illuminate why language is so powerful and also so precisely wrong in ways that formal logic is not.

The subject-predicate cut is P / G → Q

CHALKBOARD

```
Every declarative sentence has the form:

    [Subject]     [Predicate]     [Truth value]
       P              G                Q

"The cat     is on the mat"     = TRUE or FALSE
"The water    is boiling"        = TRUE or FALSE
"The policy   is working"        = ??? (the carrier is unclear)

The subject is the pattern being evaluated.
The predicate is the gradient applied to it.
The sentence is the propagation: P / G → Q.

Grammar is the gradient family: the rules for which subjects
can combine with which predicates to form valid sentences.
```

PROFESSOR LOGIC

Noam Chomsky wrote a famous sentence: "Colorless green ideas sleep furiously." It is grammatically impeccable and semantically absurd. In carrier-set terms: the sentence is a valid operation on the grammatical gradient family. Every operation stays within the rules of English syntax. But the value it tries to express is not in the semantic value space. "Colorless green" demands that something is both colourless and green simultaneously. In the semantic carrier, that is a contradiction. The grammatical operations produced a result outside the semantic carrier.

This is a closure failure in the semantic carrier, not the grammatical one. The grammar has no way to prevent it because grammar and semantics are different gradient families applied to the same value space.

What is the carrier of natural language?

CHALKBOARD

```
Classical logic claims: the carrier of all statements is {TRUE, FALSE}.

Natural language actually uses something more like:

V = { FALSE, UNLIKELY, POSSIBLE, PROBABLE, TRUE,
      ALLEGEDLY, SUPPOSEDLY, IRONICALLY TRUE,
      METAPHORICALLY TRUE, HISTORICALLY WAS TRUE,
      TRUE IN THIS CONTEXT, ... }

Plus modifiers from tense: past, present, future, conditional, subjunctive.
Plus mood: indicative, imperative, interrogative.

This is not a two-value carrier. It is a high-dimensional continuous space
with a few discrete landmarks, being approximated at the speed of speech.
```

PROFESSOR LOGIC

When a linguist says "pragmatics" they are, in carrier-set terms, talking about the context-dependence of the value space. What a sentence means depends on who said it, to whom, in what situation, with what shared knowledge. The carrier of "it is warm in here" is not {TRUE, FALSE}. It is a function of the speaker's temperature preference, the listener's expectations, the season, the social context, and at least a dozen other parameters.

This is DRAS applied to language. Every linguistic expression is a loaded pattern: a value in the semantic carrier, with a load that reflects the cost of all the contextual distinctions required to interpret it. "Pass the salt" is a low-load expression. "The policy of the previous administration was arguably effective in some respects but fundamentally misaligned with the structural constraints of the international system" is a high-load expression, and it is high-load because it is maintaining many distinctions simultaneously, all of which carry the debt from Chapter 6.

The sorites paradox: Mode 1 in language

CHALKBOARD

```
The sorites paradox (from Greek: soros = heap):

1,000,000 grains of sand is a heap. TRUE.
If n grains is a heap, then n-1 grains is also a heap. TRUE? (plausibly)
Therefore: 1 grain of sand is a heap. FALSE.

The predicate "is a heap" demands a sharp boundary in the carrier.
No sharp boundary exists in the physical world.
The predicate is trying to apply a classical {0,1} operation to a continuous
reality.

This is the same as the Liar Paradox (Mode 1):
a predicate demanding a value that is not in the carrier.

Resolution: extend V to include "borderline heap" = 0.5.
Now "is a heap" maps to {0, 0.5, 1}.
But then: is 999,998 grains a borderline heap or a borderline-borderline heap?

Higher-order vagueness.
This is Mode 2 entering language: load diverges with the depth of
qualification.
```

PROFESSOR LOGIC

The sorites paradox has occupied philosophers for 2,400 years. In carrier-set terms, it dissolves into a familiar structure: a predicate with a fuzzy boundary creates Mode 1 when the value space is {0,1}, and when you extend the value space to resolve Mode 1, you get Mode 2 (infinite regress of qualifications). This is not a philosophical problem. It is a report on the structure of language.

Language handles this by being pragmatic rather than formal. Speakers accept fuzzy predicates and manage the boundary maintenance cost through context and convention rather than formal precision. This works most of the time, at low cost, for communication. It breaks down when precision is required, exactly as the debt analysis of Chapter 6 predicts.

The distinction debt of language

CHALKBOARD

```
Every word maintains a distinction:

"cat" — distinguishes cats from non-cats
"warm" — distinguishes warm from not-warm (fuzzy boundary)
"fair" — distinguishes fair from unfair (highly context-dependent boundary)
"true" — distinguishes true from false (the core logical boundary)

Each distinction is maintained by social convention, not physics.

The cost of maintaining linguistic distinctions is paid in:
 — Dictionaries (explicit boundary records)
 — Courts (boundary disputes resolved by argument)
 — Philosophy (persistent boundary problems that have no clean resolution)
 — Misunderstanding (two people using slightly different carriers)
 — Translation failure (one language's V does not include another's values)

Language is a pragmatic carrier. Useful. Powerful. Running a constant
thermodynamic debt that is paid in disambiguation work.
```

The subject-predicate cut and formal logic

PROFESSOR LOGIC

First Order Logic (FOL) makes the subject-predicate structure explicit. A predicate P(x) is exactly the gradient G applied to the pattern x. P(x) is true when x satisfies the predicate. This is P / G → Q in the formal notation.

The difference between natural language and first-order logic is that first-order logic specifies the carrier precisely: it is {TRUE, FALSE}. Natural language does not. Natural language says "P(x) is true" and leaves the carrier implicit, context-dependent, and often fuzzy.

This is why translating natural language into formal logic is hard. You are not transcribing from one notation to another. You are taking an expression designed for a multi-valued, context-dependent, pragmatic carrier and rewriting it for a two-valued, context-free, precise carrier. Information is lost in that translation. That loss is the distinction debt, called in at the moment of formalisation.

CHALKBOARD

```
Natural language statement: "This policy is working."

Formalisation requires you to answer:
  1. What is the carrier? (TRUE/FALSE? Or a scale?)
  2. What does "working" mean? (By what measure?)
  3. For whom? (The carrier may differ by agent)
  4. At what scale? (Short-term? Long-term? Local? Global?)
  5. Compared to what? (The threshold is not in the statement)

Each answer costs work to establish.
The natural language statement deferred all of these costs to the listener.
That deferred cost is the distinction debt embedded in natural language.
```

> *"Language is logic with the debt hidden. Formal logic is language with the debt made explicit. Poetry is language that celebrates the fact that the debt can never be fully paid."*

★ **9.1** Take the sentence "The soup is hot." Identify the subject (pattern P), the predicate (gradient G), and the implicit carrier. Is the carrier {TRUE, FALSE} or something more complex? What information is lost if you force it into {TRUE, FALSE}?

Answer:

Subject P: "the soup." Predicate G: "is hot" (a function mapping soup-states to truth values). Carrier: not simply {TRUE, FALSE}. Hot is relative to: the speaker's preference, the intended use (too hot to drink? perfect for drinking? cold for soup?), the time elapsed since heating. Forcing to {TRUE, FALSE} loses all scalar and contextual information. The border between "hot" and "not hot" is maintained entirely by context, not by the sentence itself.

★★ **9.2** The sorites paradox. Is the problem with the predicate, the carrier, or the gradient family? Which paradox mode is it? Propose a minimal extension to the language carrier that resolves Mode 1, and explain why it immediately creates Mode 2.

Answer:

The problem is with the carrier (forced to be {TRUE, FALSE}) when the predicate "is a heap" has a fuzzy gradient in the real world. Mode 1: the predicate demands a value v = NOT(v) at the boundary (is the borderline case a heap or not a heap?). Resolution attempt: extend V to {0, 0.5, 1} with "borderline heap" = 0.5. Mode 2 follows: what is the status of a "borderline borderline heap"? This is the next step in self-reference: a statement about the boundary of the boundary, whose load grows without limit at each level of qualification. Higher-order vagueness is Mode 2 in the language carrier.

★★ **9.3** Chomsky's sentence: "Colorless green ideas sleep furiously." Identify the closure failure. Is this a failure of the grammatical carrier or the semantic carrier? Why can grammar not prevent it?

Answer:

The grammatical carrier ({grammatically valid sentences}) is a separate system from the semantic carrier ({meaningful statements}). The sentence is closed in the grammatical carrier: every word is the correct part of speech, every agreement is satisfied. The closure failure is in the semantic carrier: "colorless green" maps to a value outside the semantic space (contradiction in colour attribution). Grammar and semantics are two different gradient families. Grammar

checks syntactic closure. Semantics checks semantic closure. The grammatical operations have no access to the semantic value space.

★★★ **9.4** Consider the statement "I am lying right now" spoken aloud by a person. Map it onto the carrier-set framework. Is this Mode 1, Mode 2, Mode 3, or Mode 4? Does it resolve differently in spoken language than in written text? (Hint: consider the temporal carrier of the word "now".)

Answer:

Primary: Mode 1 (the Liar in natural language). But spoken language adds a temporal dimension the written Liar lacks: "now" is a specific moment. If the speaker is lying at moment t, then at moment t the statement is true (a lie is being told), which means the statement is true, which means they are NOT lying... but this reasoning takes time. The temporal carrier of spoken language means the self-reference always occurs across moments: you evaluate the statement AFTER "now" has passed. The paradox dissolves into an infinite regress (Mode 2) rather than a static contradiction (Mode 1), because each evaluation step occupies a new moment. Written text has no such temporal relief.

CHAPTER 9

Coding a Carrier

If you can write it, you understand it. Thirty lines of Python.

PROFESSOR LOGIC

Every claim in this book has been mathematical. Now I want to make it concrete in a different way: code that you can run, modify, and break.

This chapter requires Python 3. No installation of any library. No framework. Just Python and a text editor or the Python shell. If you have not coded before, this chapter will teach you everything you need as you go. Every new piece of syntax is explained when it first appears.

The goal is simple: by the end of this chapter you will have a `Carrier` class that can test any carrier you give it and report which laws hold.

Step 1: the value space as a list

CHALKBOARD: PYTHON

```
# A list in Python is written with square brackets
V = [0, 1]                  # classical carrier
V = [0, 0.5, 1]             # three-valued carrier
V = [i/100 for i in range(101)]   # 101 values from 0 to 1
```

Step 2: the operations as functions

CHALKBOARD: PYTHON

```
# lambda is Python for "a small function with no name"
# lambda v: means "given input v, return..."

NOT = lambda v: 1 - v
AND = lambda a, b: a * b
OR = lambda a, b: max(a, b)
```

Step 3: checking a law

CHALKBOARD: PYTHON

```
# all() returns True if every item in a sequence is True
# abs() gives the absolute value (distance from zero)

def check_lnc(V, AND, NOT):
    return all( abs(AND(v, NOT(v))) < 1e-10 for v in V )

def check_lem(V, OR, NOT):
    top = max(V)
    return all( abs(OR(v, NOT(v)) - top) < 1e-10 for v in V )

def check_dn(V, NOT):
    return all( abs(NOT(NOT(v)) - v) < 1e-10 for v in V )

# Test on classical carrier
V = [0, 1]
NOT = lambda v: 1 - v
AND = lambda a, b: a * b
OR = lambda a, b: max(a, b)

print(check_lnc(V, AND, NOT))   # True
print(check_lem(V, OR, NOT))   # True
print(check_dn(V, NOT))   # True
```

Step 4: finding the Liar solution

CHALKBOARD: PYTHON

```
def liar_solutions(V, NOT, tol=1e-10):
    return [v for v in V if abs(v - NOT(v)) < tol]

# Classical carrier: no solution
print(liar_solutions([0, 1], lambda v: 1-v))   # []

# Three-valued carrier: one solution
print(liar_solutions([0, 0.5, 1], lambda v: 1-v))   # [0.5]
```

Step 5: the complete Carrier class

CHALKBOARD: PYTHON

```python
# A class is a bundle of data and functions.
# self refers to "this particular carrier instance."

class Carrier:

    def __init__(self, V, not_, and_, or_, theta=1.0):
        self.V = V
        self.not_ = not_
        self.and_ = and_
        self.or_ = or_
        self.theta = theta

    def lnc(self):
        return all(abs(self.and_(v, self.not_(v))) < 1e-10
                   for v in self.V)

    def lem(self):
        top = max(self.V)
        return all(abs(self.or_(v, self.not_(v)) - top) < 1e-10
                   for v in self.V)

    def dn(self):
        return all(abs(self.not_(self.not_(v)) - v) < 1e-10
                   for v in self.V)

    def liar(self):
        return [v for v in self.V
                if abs(v - self.not_(v)) < 1e-10]

    def report(self):
        print(f"V = {self.V[:6]}{'...' if len(self.V)>6 else ''}")
        for name, fn in [("LNC", self.lnc),("LEM", self.lem),("DN", self.dn)]:
            mark = "✓" if fn() else "✗"
            print(f" {mark} {name}")
        sols = self.liar()
        if sols: print(f" Liar resolved at {sols}")
        else: print(" Liar: paradox (no fixed point)")

# Use it
c = Carrier([0, 0.5, 1], lambda v: 1-v, lambda a,b: a*b, lambda a,b: max(a,b))
c.report()
```

CHAPTER 9 EXERCISES

★ **9.1** Run the Carrier class on the classical {0,1} and paraconsistent {0,0.5,1} carriers. Verify the output matches your hand calculations from Chapters 3 and 6.

★★ **9.2** Add a `closure_check` method that tests whether NOT maps every value in V back to a value in V. Test it on the standard carriers and on one where you deliberately break closure by returning 2.0 for NOT(1).

Answer:

def closure_check(self): V_set = set(round(v,10) for v in self.V); return all(round(self.not_(v),10) in V_set for v in self.V). A NOT that returns 2.0 for value 1 will fail: 2.0 is not in {0, 0.5, 1}.

★★ **9.3** Build the carrier from your Chapter 7 design exercise and run `report()` on it. Does the output match your hand derivation?

★★★ **9.4** Write a function `compare(c1, c2)` that takes two Carrier instances and prints a table showing which laws each has in common and which differ. Test it on classical versus paraconsistent.

Answer:

def compare(c1, c2): checks = [("LNC",lambda c:c.lnc()),("LEM",lambda c:c.lem()), ("DN",lambda c:c.dn()),("Liar",lambda c:bool(c.liar()))]; for name,fn in checks: r1,r2=fn(c1),fn(c2); changed=" <-- CHANGED" if r1!=r2 else ""; print(f"{name}: {r1} vs {r2}{changed}")

CHAPTER 10

Your Challenge

Design a formal system. Verify it. Defend it.

PROFESSOR LOGIC

This is the final chapter, and it is the most open. There is no single correct answer. There is only a standard: every claim must be verified, either by hand or by code. If you cannot verify it, it is not finished.

That standard is the empirical standard. It is what separates a formal system from a philosophical opinion. Your carrier can be about anything. Your operations can be unconventional. Your threshold can be unusual. But every law you claim to hold must be demonstrated, value by value, with the result shown.

CHALKBOARD: THE REQUIREMENTS

1. Carrier specification
State V. State your encoding choices. State which values are designated.

2. Operations
Define NOT, AND, OR for every value. Prove closure.

3. Forced laws
Check LNC, LEM, DN. State at least two additional laws your carrier forces. State at least two laws that are NOT forced, with counterexamples.

4. Load and threshold
State θ. Give one example pattern that coheres and one that does not.

5. Failure modes
State two situations where your system breaks. Name the mode (1, 2, 3, 4).

6. Utility
When is this logic better than classical {0,1}? What does it model that classical cannot?

7. Verification (required)
Either hand-verify all claims, or run the Carrier class from Chapter 9. Include the output.

Suggested domains

Domain	Why it is interesting
Medical diagnosis	Partial evidence, competing hypotheses, threshold effects
Environmental risk	Continuous values, irreversibility, multi-scale measurements
Legal evidence	Weight of evidence, burden of proof, categorical distinctions
Financial modelling	Probability over outcomes, resource constraints, time-dependence
Social trust	Relational values, transitivity, history-dependence
Game states	Partial information, simultaneous action, payoff structure
Ecological capacity	Resource limits, thresholds, multi-species interaction

PROFESSOR LOGIC

One final note on the distinction operation idea raised in the preface. A distinction operation D(a, b) would measure the cost of telling two values apart. In a carrier where all values are equally spaced, D might simply be |a − b|. In a carrier with non-uniform spacing, the cost of distinguishing nearby values might be higher.

This connects to load: maintaining a distinction between nearly-equal values requires more work than maintaining a distinction between clearly different values. D(0.49, 0.51) should cost more than D(0, 1). A formal distinction operation could be D(a, b) = |a − b| / (max(V) − min(V)): normalised distance in the value space. I leave this as an open thread. If you formalise it in your Chapter 10 project, send it along. Good ideas deserve company.

"Every formal system is a choice. The choice of what values to allow. The choice of how to combine them. The choice of how much to tolerate. Change any one and a different world of consequences becomes forced. Make your choices deliberately. Verify what they force. That is the whole of the discipline."

ANSWER KEY

Selected Answers

Detailed answers are embedded in each chapter. These are the core results.

PROFESSOR LOGIC

Every chapter exercise has a "show answer" button inline. Click it and you get the working, not just the result. This section summarises the key results across chapters for quick reference when you are reviewing.

Law	{0,1}	{0,0.5,1}	[0,1] sampled
LNC	✓ forced	✗ fails at 0.5	✗ fails at interior
LEM	✓ forced	✗ fails at 0.5	✗ fails at interior
Double Negation	✓ forced	✓ forced	✓ forced
Liar solution	None	v = 0.5	v = 0.5

CORE FORMULAS

```
NOT(v) = 1 − v     AND(a,b) = a × b     OR(a,b) = max(a,b)

Load rules:   NOT: unchanged    AND: adds    OR: takes minimum

Liar fixed point:   v = NOT(v)  →  v = 1 − v  →  2v = 1  →  v = 0.5

Three parameters:   V (values)    Γ (operations)    θ (threshold)

Four paradox modes:   Liar    Gödel    Russell    Yablo
```